

MANI — Documento de Diseño (DD) V2

Empresa: TRAMA · Ingeniería de Software **Producto:** MANI — plataforma multi-tenant de formalización de operaciones de servicio **Documento:** DD V2 (Entrega 4) **Estado:** Borrador para revisión **Fecha:** 2026-09-23 **Fuentes:** Product/DD-MANI.md (V4 interna), SRS V3, SAD V3, SDD V1, Documento de Infraestructura V1, Product/Modelo_Datos_MANI.md, Product/DDL_MANI.sql, MANI-Flutter/database/migrations 001-007, PoC-001..004, Inf_test-002

Historial de versiones

Versión	Momento	Cambios principales
V1 (Entrega 3)	Sprint 2, 2026-09-03	Vista de componentes, API por módulo, 3 secuencias críticas, errores e idempotencia, RLS, despliegue, trazabilidad a pruebas y RF → entidad → ADR. (Internamente V1-V4 del 03/09 al 16/09.)
V2 (Entrega 4)	Sprint 2 Review, 2026-09-23	Componentes: reparto Core / Reglas (Java) / Despacho (.NET) + API Gateway (SDD V1). API: columna de servicio destino; rutas /rechazar y /aliados-validos; /tenants/resolve. Datos: valores de dominio normalizados (migración 007); 2 índices de cobertura (PoC-004); inventario de migraciones. Seguridad: RLS con restricción de rol (corrige H-02); política KYC corregida (H-01, H-02, H-04). Despliegue: §9 alineada con DOC-08 e Infraestructura V1 (corrige "Flutter no se conteneriza"). Correcciones: nota de KI-02 apunta a ADR-0023 (no 0021). Glosario: se integra en este documento (§14) el antiguo Product/Glosario_Terminos_MANI.md, que queda como única fuente de definiciones del proyecto.

Índice

- [1. Propósito y alcance](#)
- [2. Trazabilidad SRS → SAD → SDD → DD](#)
- [3. Vista de componentes](#)
- [4. Modelo de datos](#)
- [5. Diseño de API por módulo](#)
- [6. Diagramas de secuencia](#)
- [7. Manejo de errores e idempotencia](#)
- [8. Diseño de seguridad](#)
- [9. Vista de despliegue](#)
- [10. Trazabilidad hacia pruebas](#)
- [11. Trazabilidad RF → entidad → servicio → ADR](#)
- [12. Fuera de alcance](#)
- [13. Convenciones](#)
- [14. Glosario de términos](#)

1. Propósito y alcance

Especifica el **cómo** a nivel de diseño de solución: componentes, contrato de API, flujos críticos, errores, seguridad y despliegue que un desarrollador necesita para implementar el MVP (EP-01..EP-

06). No repite decisiones del SAD ni de los ADR; no repite el diccionario de datos (Modelo_Datos_MANI.md). No es un contrato OpenAPI ejecutable: fija recurso, verbo, rol, servicio y RF.

Alcance: RF-01..RF-23. RF-24..RF-28 en §12, sin diseñar.

2. Trazabilidad SRS → SAD → SDD → DD

SRS (qué) → SAD (drivers, killers, QS, trade-offs) → SDD (vistas, contenedores, validación) → DD (API, secuencias, errores, RLS, despliegue) + Modelo de Datos → ADR (por qué)

Consume: SRS V3 (RF-01..23, RNF-01..11, REST-01..05); SAD V3 §7 y §8; SDD V1 §5–§13; Modelo_Datos; ADR-0004, 0006, 0011, 0012, 0013, 0015, 0016, 0017, 0018, 0021, 0022, 0023 y los borradores 0027/0028.

3. Vista de componentes

Capacidad	Módulo	Servicio	Componente técnico	ADR
Tenants y aislamiento	M-01	Core	Serverpod + Postgres/RLS	0012, 0018
Autenticación	M-01	Supabase Auth + Core	IdP + hook de claims + middleware JWT en cada servicio	0018, 0022
Directorio aliados / KYC	M-02	Core	Storage kyc-documentos , ruta tenant_id/<uid>/...	0013
Requisitos KYC por tenant	M-02	Reglas (Java)	Evaluador KYC	0028 🟡
Clientes y sitios	M-03	Core	Postgres	0012
Catálogo y cobertura	M-04	Core	Catálogo de zonas global, solo lectura	0011
Despacho y asignación	M-05	Despacho (.NET)	RPC atómicas crear_solicitud , aliados_validos , aceptar_solicitud , rechazar_solicitud	0016, 0021, 0028 🟡
Orden del listado	M-05	Reglas (Java)	Evaluador de ranking	0028 🟡
Cotización y ajuste	M-06	Core (+ Reglas para RF-16)	cotizacion versionada	—
Ejecución (log)	M-07	Core	evento_servicio append-only	RNF-04
Calificación	M-08	Core	calificacion con UNIQUE	RF-19
Mensajería	M-09	Core + Supabase Realtime + FCM/APNs	Broadcast + push	0017
Tarifario y reportes	M-11	Core (+ Reglas lee tarifas)	tarifa_referencia	—
Entrada única	—	API Gateway	Spring Cloud Gateway	0027 🟡

✅ **KI-02 cerrado por ADR-0023** (no ADR-0021, corrección de V1): Java, .NET y Serverpod/Supabase coexisten gobernando módulos distintos; se retira Azure.

4. Modelo de datos

El modelo lógico (entidades, diccionario, cardinalidades) vive en `Product/Modelo_Datos_MANI.md`; el físico en `Product/DDL_MANI.sql`. Regla de aislamiento: toda entidad tenant-scoped lleva `tenant_id` protegido por RLS; excepciones `tenant` y `zona`.

4.1 Cadena de migraciones (fuente ejecutable)

#	Migración	Contenido
001	<code>initial_schema</code>	Esquema base (17 tablas, <code>pgcrypto</code> , FK)
002	<code>verificacion_aliados</code>	RPC y estados de verificación (US-02.1.3)
003	<code>categorias_servicio</code>	Categorías + <code>CHECK</code> de estado (<code>NOT VALID</code>)
004	<code>aliado_categorias</code>	Relación aliado-categoría, índices únicos
005	<code>aceptar_rechazar_solicitud</code>	RPC de aceptación/rechazo, tabla <code>solicitud_rechazo</code>
006	<code>crear_solicitud</code>	RPC de creación, tabla <code>solicitud_foto</code> , bucket <code>solicitudes</code>
007	<code>normalizar_dominios</code>	Normaliza datos a MAYÚSCULA en español; valida los <code>CHECK</code> de 003
008 ●	<code>rls_tenant_isolation</code>	Pendiente: versionar las 16 políticas <code>tenant_isolation_*</code> con restricción de rol (§8.1) — SCRUM-1051
009 ●	<code>indices_cobertura</code>	Pendiente: 2 índices de §4.3 — SCRUM-1054
010 ●	<code>promover_poc</code>	Pendiente: hook de claims (CFG-12), bucket <code>kyc-documentos</code> + <code>kyc_isolation</code> (CFG-13), RPC de cobertura

`Product/DDL_MANI.sql` está desactualizado respecto de QA (PoC-001, Inf_PoC-001): hasta regenerarlo, **la cadena de migraciones es la fuente de verdad**.

4.2 Valores de dominio (migración 007)

Campo	Valores en BD	En el JWT
<code>usuario.rol</code>	<code>ADMIN_PLATAFORMA</code> , <code>ADMIN_TENANT</code> , <code>ALIADO</code> , <code>CLIENTE</code>	minúscula (<code>lower(rol)</code>) — contrato de ADR-0018
Estado de entidades	<code>ACTIVO</code> , <code>INACTIVO</code>	—
<code>aliado.estado_verificacion</code>	<code>PENDIENTE</code> , <code>VERIFICADO</code> , <code>RECHAZADO</code>	—
<code>solicitud.estado</code>	<code>PENDIENTE</code> , <code>ASIGNADA</code> , ...	—

● `solicitud.estado` aún sin `CHECK/DEFAULT` en QA (deuda de PoC-001) — incluir en 008.

4.3 Índices obligatorios (PoC-004)

```
CREATE INDEX IF NOT EXISTS ix_cobertura_aliado_tenant_zona
ON cobertura_aliado (tenant_id, zona_id);
CREATE INDEX IF NOT EXISTS ix_aliado_categoria_tenant_categoria
ON aliado_categoria (tenant_id, categoria_id);
```

Efecto medido a 100k aliados por tenant: p95 123,91 ms → 9,48 ms (umbral 50 ms).

5. Diseño de API por módulo

Prefijo `/api/v1` (omitido). Toda operación autenticada exige `Authorization: Bearer <JWT>;` ninguna acepta `tenant_id` en body ni cabecera editable. Columna **Servicio** = destino en el gateway (SDD V1 §9.4).

5.1 M-01 — Plataforma y acceso

Método	Recurso	Rol	Servicio	RF	Notas
GET	<code>/tenants/resolve?slug=</code>	público	Core	RF-03	Pre-auth (ADR-0018 fase 1); devuelve <code>tenant_id</code> , nombre, marca solo si <code>ACTIVO</code>
POST	<code>/tenants</code>	<code>admin_plataforma</code>	Core	RF-01	Crea el tenant aislado desde la primera fila
PATCH	<code>/tenants/:id/estado</code>	<code>admin_plataforma</code>	Core	RF-01	Activa/desactiva
—	Login	público	Supabase Auth (SDK)	RF-03	<code>signInWithPassword</code> ; el hook inyecta claims
—	Recuperar contraseña	público	Supabase Auth (SDK)	RF-04	Enlace < 15 min (QS-03)

5.2 M-02 / M-03 — Directorio

Método	Recurso	Rol	Servicio	RF	Notas
POST	<code>/aliados</code>	<code>aliado</code>	Core	RF-05	<code>tipo</code> : <code>persona_natural</code> / <code>empresa</code> / <code>empleado_directo</code>
GET	<code>/reglas/requisitos-kyc?tipoAliado=</code>	<code>aliado</code>	Reglas	RF-05, RNF-10	Documentos exigidos por el tenant
POST	<code>/aliados/:id/documentos</code>	<code>aliado</code>	Core	RF-05	Sube a <code>tenant_id/<uid>/...</code> con la utilidad única de rutas (§8.2)
PATCH	<code>/aliados/:id/verificacion</code>	<code>admin_tenant</code>	Core	RF-06	VERIFICADO / RECHAZADO
POST	<code>/aliados/:id/cobertura</code>	<code>aliado</code>	Core	RF-07	Sobre catálogo <code>zona</code>
POST	<code>/clientes</code>	<code>cliente</code>	Core	RF-08	<code>persona_natural</code> / <code>empresa</code>
POST	<code>/clientes/:id/sitios</code>	<code>cliente</code>	Core	RF-09	Rechaza sin <code>zona_id</code>

5.3 M-04 — Catálogo

Método	Recurso	Rol	Servicio	RF
POST	<code>/categorias</code>	<code>admin_tenant</code>	Core	RF-10
PATCH	<code>/categorias/:id/estado</code>	<code>admin_tenant</code>	Core	RF-10
POST	<code>/aliados/:id/categorias</code>	<code>aliado</code>	Core	RF-11
POST	<code>/categorias/:id/tarifas</code>	<code>admin_tenant</code>	Core	RF-22

5.4 M-05..M-08 — Ciclo del servicio

Método	Recurso	Rol	Servicio	RF	Notas
POST	/solicitudes	cliente	Despacho	RF-12	Idempotency-Key ; valida zona del sitio
GET	/solicitudes/:id/aliados-validos	cliente	Despacho (+ Reglas)	RF-12, RF-13	Match zona × cobertura × categoría; orden por Reglas; paginado (PoC-004 H-03)
POST	/solicitudes/:id/aceptar	aliado	Despacho	RF-14	UPDATE condicional; 409 si tomada (§7.1)
POST	/solicitudes/:id/rechazar	aliado	Despacho	RF-14	Registra en solicitud_rechazo
POST	/solicitudes/:id/cotizaciones	aliado	Core (+ Reglas)	RF-15, RF-16	Alerta si fuera de rango
POST	/cotizaciones/:id/aceptar · /rechazar · /solicitar-ajuste	cliente	Core	RF-17	Ajuste → nueva version
GET	/solicitudes/:id/eventos	cliente, aliado, admin_tenant	Core	RF-18	Append-only
POST	/solicitudes/:id/calificaciones	cliente, aliado	Core	RF-19	Una por autor

5.5 M-09 — Comunicación

Método	Recurso	Rol	Servicio	RF
POST	/solicitudes/:id/mensajes	cliente, aliado	Core	RF-20
GET	/solicitudes/:id/mensajes	cliente, aliado, admin_tenant	Core	RF-21

5.6 M-11 — Tarifario

Método	Recurso	Rol	Servicio	RF
GET	/reportes/cotizaciones-fuera-de-rango?desde=&hasta=	admin_tenant	Core	RF-23

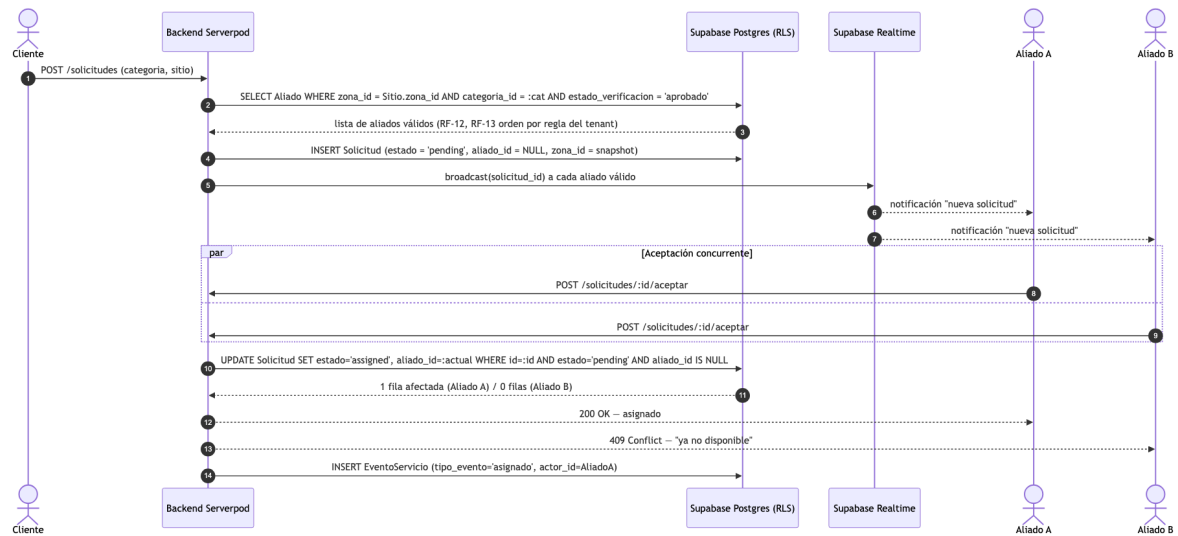
5.7 Endpoints internos (no expuestos en el gateway)

Método	Recurso	Llamador	Destino
POST	/internal/v1/notificaciones	Despacho	Core
POST	/api/v1/reglas/ranking	Despacho	Reglas
POST	/api/v1/reglas/cotizaciones/validar	Core	Reglas
GET	/health, /metrics	Plataforma	Todos (red privada)

6. Diagramas de secuencia

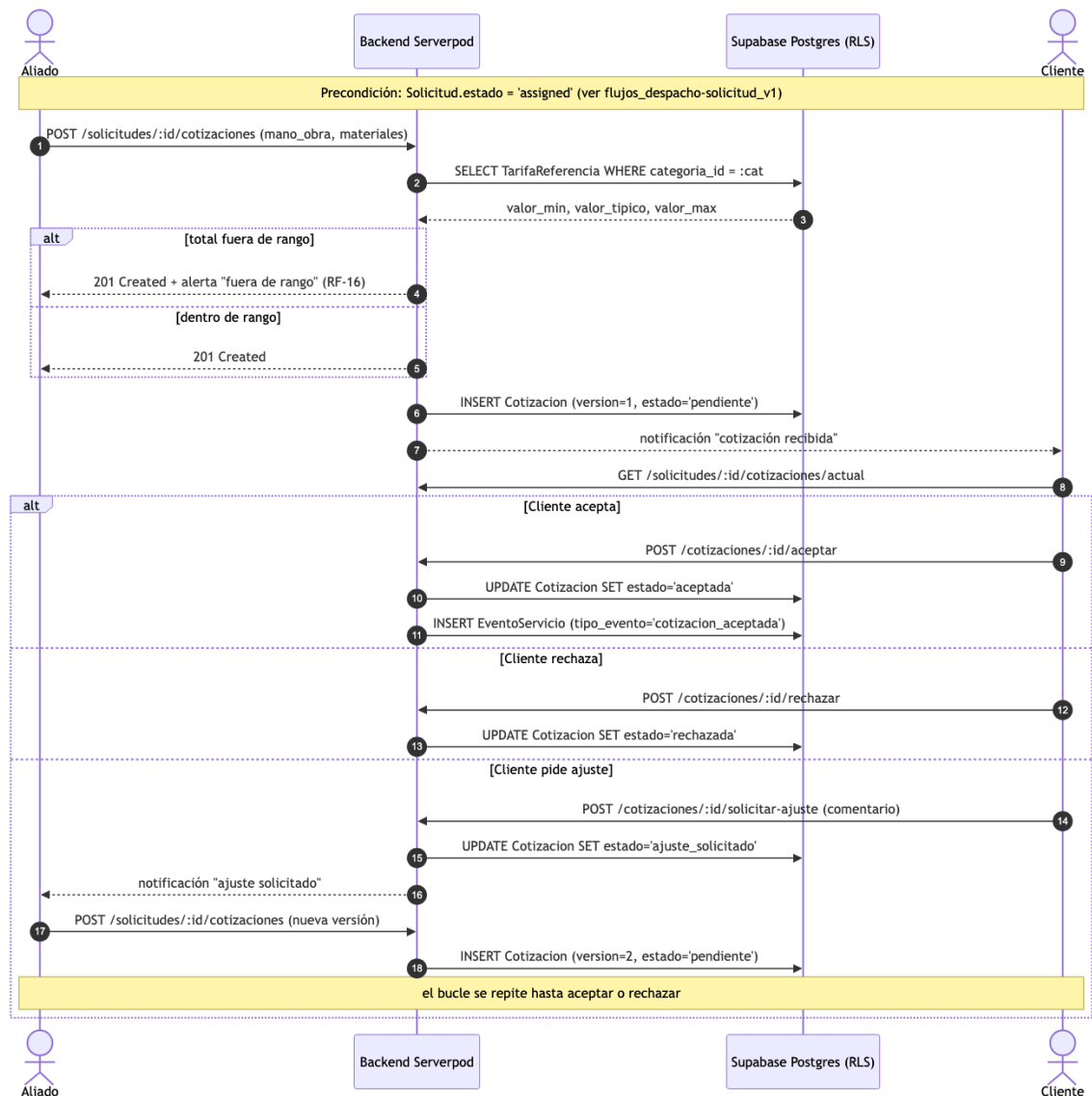
Fuente versionada como texto (ADR-0008): Diagramas/Negocio/flujo/*.mmd. Las versiones con el reparto de servicios (Gateway / Despacho / Reglas / Core) están en el SDD V1 §6.

6.1 Despacho (broadcast + asignación atómica) – RF-12..RF-14, RNF-05



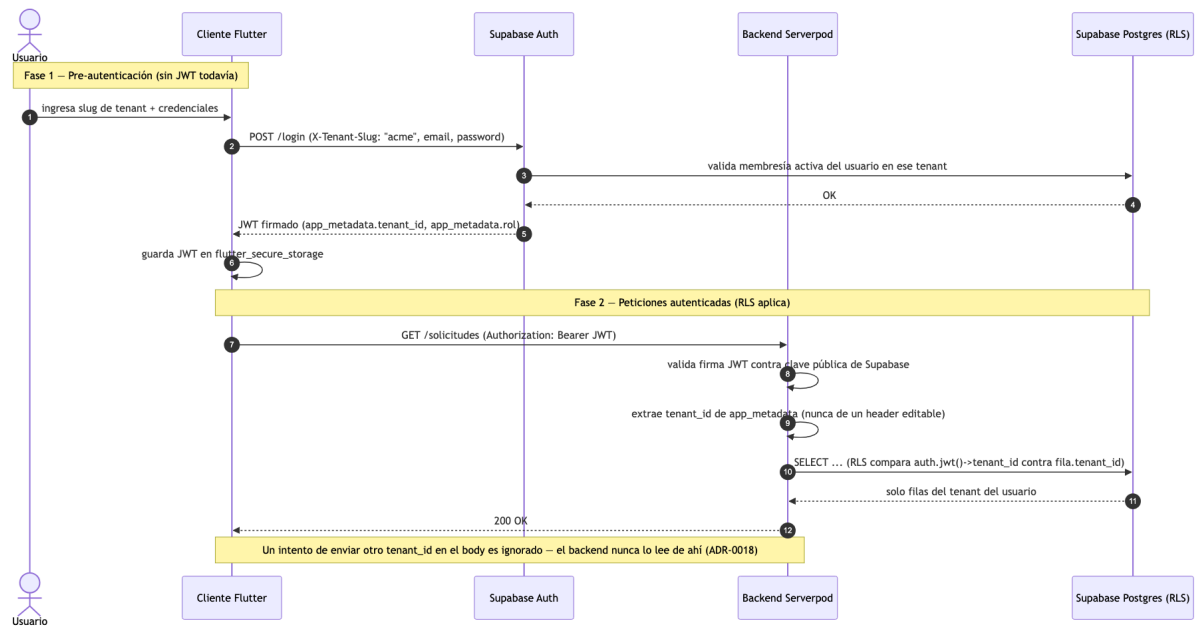
Broadcast a **todos** los aliados válidos a la vez; asignación con un único `UPDATE` condicional; el cliente del aliado espera la respuesta del servidor antes de mostrar "asignado a ti". ● Falta el `.mmd` fuente de este diagrama.

6.2 Cotización con ajuste — RF-15..RF-17



Cada ajuste crea una **nueva versión** de cotización; el historial queda para auditoría (RNF-04).

6.3 Resolución de tenant y autenticación — RNF-01



El `tenant_id` existe para el backend solo dentro del JWT verificado (`app_metadata.tenant_id`). `X-Tenant-Slug` solo antes del JWT. El claim lo inyecta `custom_access_token_hook` (PoC-002), *fail-closed*: sin fila en `usuario`, sin claims.

7. Manejo de errores e idempotencia

7.1 Aceptación de solicitud (RF-14, RNF-03, RNF-05)

Escenario	HTTP	Respuesta	Evidencia
Primer aliado en aceptar	200	solicitud con estado='ASIGNADA', aliado_id propio	PoC-001 (1×200)
Otro aliado ya la tomó	409	{ "error": "ya_no_disponible" }	PoC-001 (49×409)
El mismo ganador reintent	200 idempotente	Mismo resultado; 0 filas nuevas en bitácora	PoC-001 r5
Inexistente o de otro tenant	404	No distingue "no existe" de "no es tuya"	PoC-001 r5
Usuario sin rol aliado	403	{ "error": "rol_no_autorizado" }	🟡 Nuevo — corrige H-02

7.2 Calificación (RF-19, RNF-03)

Escenario	HTTP	Respuesta
Primera calificación del autor	201	Recurso creado
Reintento del mismo autor	200	Recurso existente (captura de <code>UNIQUE(solicitud_id, autor_id)</code> , nunca 500)
Servicio sin ejecución finalizada	422	Solo válida tras RF-18 completado

7.3 Regla general

Toda escritura de §5.4 acepta `Idempotency-Key` (obligatorio en aceptar, cotizar y calificar); el receptor guarda la respuesta por clave 24 h. Complementa —no reemplaza— las garantías de base de datos (condicionales y `UNIQUE`).

7.4 Códigos de error de dominio

Prefijo `MANI-<MÓDULO>-<HTTP>`, ya usado por las RPC (`Inf_test-002`): p. ej. `MANI-SOL-403`, `MANI-CAT-403`. PostgreSQL los expone como `PT409` / `PT404`. Cada servicio traduce a HTTP y cuerpo `{ "error": "<codigo_legible>", "code": "MANI-..." }`.

7.5 Timeouts y degradación

Gateway → backend 5 s; backend → backend 2 s; sin reintentos automáticos en operaciones no idempotentes. Si Reglas no responde, Despacho usa orden por defecto (cobertura) y registra el evento degradado.

8. Diseño de seguridad

8.1 Aislamiento por RLS con restricción de rol (RNF-01, ADR-0012, ADR-0018)

Patrón V1 (vigente en QA) — aísla tenant, **no rol** (H-02):

```
CREATE POLICY tenant_isolation_solicitud ON solicitud
  USING (tenant_id = (auth.jwt() -> 'app_metadata' ->> 'tenant_id')::uuid);
```

🟡 **Patrón V2 propuesto** (migración 008): lectura por tenant, escritura acotada por rol y función auxiliar que centraliza el acceso a `auth.jwt()` (facilita TO-10 / QS-22):

```
CREATE OR REPLACE FUNCTION app.tenant_id() RETURNS uuid
  LANGUAGE sql STABLE AS
  $$ SELECT (auth.jwt() -> 'app_metadata' ->> 'tenant_id')::uuid $$;

CREATE OR REPLACE FUNCTION app.user_role() RETURNS text
  LANGUAGE sql STABLE AS
  $$ SELECT auth.jwt() -> 'app_metadata' ->> 'user_role' $$;

ALTER TABLE solicitud ENABLE ROW LEVEL SECURITY;

CREATE POLICY solicitud_select ON solicitud
  FOR SELECT TO authenticated
  USING (tenant_id = app.tenant_id());

CREATE POLICY solicitud_insert_cliente ON solicitud
  FOR INSERT TO authenticated
  WITH CHECK (tenant_id = app.tenant_id() AND app.user_role() = 'cliente');

CREATE POLICY solicitud_update_aliado ON solicitud
  FOR UPDATE TO authenticated
```

```
USING (tenant_id = app.tenant_id() AND app.user_role() = 'aliado')
WITH CHECK (tenant_id = app.tenant_id());
```

Además, la RPC `aceptar_solicitud` (`SECURITY DEFINER`) debe verificar `app.user_role() = 'aliado'` antes del `UPDATE`, porque `SECURITY DEFINER` omite RLS.

Validación obligatoria: agregar a `mani-aislamiento` el caso "cliente intenta aceptar" con su espejo positivo (lección del control negativo de PoC-002).

8.2 Aislamiento de documentos KYC (ADR-0013, corregido por PoC-003)

```
-- Lectura: el propio aliado o el admin del mismo tenant
CREATE POLICY kyc_select ON storage.objects FOR SELECT TO authenticated
USING (
  bucket_id = 'kyc-documentos'
  AND (storage.foldername(name))[1] = app.tenant_id()::text
  AND (
    (storage.foldername(name))[2] = auth.uid()::text
    OR app.user_role() = 'admin_tenant'
  )
);

-- Escritura: solo el propio aliado en su carpeta (H-04: admin_tenant ya no escribe ni borra)
CREATE POLICY kyc_insert ON storage.objects FOR INSERT TO authenticated
WITH CHECK (
  bucket_id = 'kyc-documentos'
  AND (storage.foldername(name))[1] = app.tenant_id()::text
  AND (storage.foldername(name))[2] = auth.uid()::text
);
```

Correcciones incorporadas:

Hallazgo	Corrección
H-01 URL firmada = token al portador	TTL ≤ 60 s; emisión solo desde el Core
H-02 aliado_id vs auth.uid()	La ruta usa <code>auth.uid()</code> (id de usuario), no <code>aliado.id</code>
H-04 FOR ALL	Separada en <code>SELECT</code> e <code>INSERT</code> ; <code>admin_tenant</code> solo lee (pendiente de ratificar en Mesa, Infra M-04)

Corolario (KI-05): la ruta `tenant_id/<uid>/archivo` se construye en **una única función** del Core, nunca por endpoint.

8.3 Control de acceso por rol (dos capas)

1. **RLS (datos):** tenant siempre; rol en escrituras críticas (§8.1).
2. **Middleware de autorización (cada servicio):** rol por endpoint (§5). Ej.: solo `admin_tenant` en `PATCH /aliados/:id/verificacion`.

Ambas son necesarias: sin middleware, cualquier usuario del tenant aprobaría aliados; sin RLS, se repite el riesgo que ADR-0012 descartó.

8.4 Reglas vinculantes

`service_role` solo en el Core; `SECURITY DEFINER` toma el tenant de `auth.jwt()`; ningún endpoint lee `tenant_id` del body; logs con `tenant_id` y sin datos personales; JWKS cacheado (PoC-002: 160 ms en frío).

9. Vista de despliegue

Corrige la V1 (que asumía "namespace de Kubernetes por ambiente" y "Flutter no se containeriza").
Detalle completo en Documento de Infraestructura V1.

Ambiente	Rama	App	Backends	Supabase
DEV	develop	Docker Compose <code>mani-web</code> :8080 (Nginx)	Compose local	<code>postgres:16-alpine</code> local
QA	release	<code>flutter-web-staging.zip</code> · imagen :staging	Railway Staging	Proyecto QA (001-007 aplicadas)
PROD	main	Imagen GHCR :latest / :vX.Y.Z + tiendas (móvil)	Railway Production	Proyecto PROD

- **Flutter Web sí se containeriza** (Dockerfile multi-stage → `nginx:alpine`); la app móvil se distribuye por tiendas.
- **Kubernetes (PROY-08)**: clúster de referencia k3d/kind con las mismas imágenes (ADR-0029 borrador).
- **Observabilidad (ADR-0006)**: cada servicio expone `/health` y `/metrics` en red privada, exentos de token de tenant. `evento_servicio` (log de negocio) no sustituye al log operacional.

10. Trazabilidad hacia pruebas

Elemento	Qué probar	Referencia	Estado
§7.1 Despacho atómico	Exactamente 1 asignación con N concurrentes	QS-09	PoC-001 (k6)
§7.1 Rol en aceptación	<code>cliente</code> no puede aceptar	H-02	caso nuevo en Newman
§6.3 Resolución de tenant	0 filas ajenas tras alterar headers/JWT	ADR-0015, QS-02	PoC-002
§7.1 Idempotencia	Reintento sin efecto duplicado	QS-11	PoC-001 r5
§7.2 Calificación	Doble toque = 1 fila	QS-13	
§8.2 Storage	Aliado no lee KYC de otro; admin no escribe	ADR-0013, KI-05	caso 6 (95/95) · caso H-04
§4.3 Índices	p95 < 50 ms a 100k	QS-08	PoC-004 (con índices)
§4.1 Migraciones	Idempotentes, dos pasadas	README de migraciones	Inf_test-002
Pruebas Flutter	313 pruebas, 18 de integración, cobertura 85,4 %	CI	Inf_test-002

11. Trazabilidad RF → entidad → servicio → ADR

RF	Entidad(es)	Servicio	ADR
RF-01	tenant	Core	—
RF-02	tenant (config), categoria_servicio, tarifa_referencia	Core + Reglas	0014, 0028 ●
RF-03	usuario	Auth + Core	0018, 0022
RF-04	usuario	Auth	0022
RF-05	aliado, documento_kyc	Core + Reglas	0013
RF-06	aliado.estado_verificacion, documento_kyc	Core	0013
RF-07	cobertura_aliado, zona	Core	0011
RF-08	cliente	Core	—
RF-09	sitio	Core	0011
RF-10	categoria_servicio	Core	—
RF-11	aliado_categoria	Core	—
RF-12	solicitud, cobertura_aliado, aliado_categoria, solicitud_foto	Despacho	0011, 0028 ●
RF-13	orden calculado (no persistido)	Reglas	0028 ●
RF-14	solicitud.estado, solicitud.aliado_id, solicitud_rechazo	Despacho	0016, 0021
RF-15	cotizacion	Core	—
RF-16	cotizacion, tarifa_referencia	Core + Reglas	0028 ●
RF-17	cotizacion.estado, cotizacion.version	Core	—
RF-18	evento_servicio	Core	—
RF-19	calificacion	Core	—
RF-20	mensaje, notificacion	Core + Realtime	0017
RF-21	mensaje	Core	0017
RF-22	tarifa_referencia	Core	—
RF-23	cotizacion + tarifa_referencia	Core	—

12. Fuera de alcance

- **2º incremento (RF-24..RF-28):** pago, liquidación, queja, comercialización, métricas. Dependen del operador de pagos (SRS §3.2, REST-03).
- **Contrato OpenAPI ejecutable:** se genera por servicio en implementación (requisito de ZAP).
- **Particionamiento:** sin datos reales que lo justifiquen; los índices de §4.3 sí entran.
- **Dimensionamiento de nodos K8s:** SP-TO-11 / ADR-0029.

13. Convenciones

- PK `id` (UUID); FK `<entidad>_id`; `tenant_id` primera columna tras `id` en tablas tenant-scoped.
- Fechas `created_at` / `updated_at` en UTC.
- Valores de dominio en MAYÚSCULA en español en la BD; rol en minúscula en el JWT (§4.2).
- Rutas en `kebab-case`, recursos en plural.
- Migraciones numeradas `NNN_descripcion.sql`, idempotentes.

14. Glosario de términos

Integra el contenido completo de `Product/Glosario_Terminos_MANI.md` (secciones 14.1 a 14.3, sin cambios en el texto). Las correcciones y los términos nuevos van marcados con  en 14.4 a 14.7. Este glosario es la única fuente de definiciones del proyecto: SRS, SAD, SDD e Infraestructura lo referencian.

Este documento centraliza las definiciones de negocio y términos técnicos utilizados en la documentación, backlog y arquitectura del ecosistema MANI.

14.1 Términos de negocio

- **Aliado:** Proveedor independiente o empresa registrada en la plataforma que ejecuta los servicios locativos (ej. plomeros, electricistas).
- **Cliente:** Usuario final (persona natural o jurídica) que solicita y paga por un servicio locativo a través de la plataforma.
- **Backoffice:** Interfaz o consola administrativa utilizada por el personal interno de la plataforma para gestionar la operación (aprobación de aliados, revisión de PQR, configuración de tarifas).
- **Ciclo del Servicio:** Flujo completo de una solicitud, que abarca desde la petición inicial del cliente, la cotización, la aceptación, la ejecución, hasta el pago y la calificación final.
- **Cobertura:** Delimitación geográfica (zonas o radio en kilómetros) dentro de la cual un Aliado específico ofrece sus servicios.
- **Tenant (Inquilino):** En el contexto de negocio, representa a una empresa, marca o franquicia que utiliza la plataforma como su propio sistema (Marca Blanca), operando de forma aislada de otros inquilinos.

14.2 Términos técnicos y de arquitectura

- **Multi-tenant (Multi-inquilino):** Arquitectura de software donde una única instancia de la aplicación se ejecuta en el servidor y sirve a múltiples Tenants. Los datos están centralizados

pero estrictamente aislados por seguridad.

- **Microservicios:** Patrón de arquitectura donde el backend de la aplicación se divide en pequeños servicios independientes (ej. Servicio de Identidad, Servicio de Pagos), cada uno con su propia lógica y base de datos lógica.
- **API Gateway (Puerta de enlace API):** Componente que actúa como un único punto de entrada para el frontend (app móvil o web). Recibe las peticiones del usuario y las redirige al microservicio correspondiente.
- **RLS (Row-Level Security):** Política de seguridad implementada directamente en el motor de la base de datos (PostgreSQL). Asegura que cada fila de datos solo pueda ser consultada o modificada por el usuario o Tenant que tiene los permisos adecuados, previniendo fugas de datos.
- **JWT (JSON Web Token):** Estándar abierto utilizado para transmitir información de sesión de forma segura entre el frontend y el backend (Identity Provider).
- **Identity Provider / IdP (Proveedor de Identidad):** Servicio externalizado (como Supabase Auth) encargado exclusivamente de gestionar el registro, inicio de sesión y validación de usuarios.
- **BaaS (Backend as a Service):** Modelo en la nube que proporciona servicios backend listos para usar (como bases de datos, autenticación y almacenamiento). En MANI se utiliza Supabase.
- **Blob Storage / Object Storage:** Servicio de almacenamiento en la nube diseñado para guardar archivos no estructurados (fotos, documentos PDF).
- **PostGIS:** Extensión de la base de datos PostgreSQL que permite almacenar y realizar consultas geográficas complejas (ej. buscar aliados cercanos por GPS).

14.3 Términos de metodología (Agile / Scrum)

- **Spike Técnico:** Historia de usuario orientada puramente a la investigación y experimentación. Se utiliza cuando el equipo necesita resolver una duda técnica compleja antes de poder estimar y desarrollar una funcionalidad.
- **MVP (Minimum Viable Product):** Producto Mínimo Viable. La versión inicial del sistema con las funcionalidades estrictamente necesarias (Core) para salir al mercado y aportar valor al usuario.
- **Mockup:** Representación visual estática y de alta fidelidad de la interfaz de usuario.
- **BDD (Behavior-Driven Development):** Metodología de desarrollo donde los criterios de aceptación se escriben en un lenguaje natural estructurado (Given / When / Then) para que sean comprensibles tanto por el negocio como por los programadores.
- **INVEST:** Acrónimo (Independiente, Negociable, Valiosa, Estimable, Pequeña, Testable) utilizado como lista de verificación para garantizar la calidad en la redacción de las Historias de Usuario.



14.4 Correcciones a definiciones anteriores

Prevalece la definición corregida; el texto original de 14.1 a 14.3 se conserva para no perder el historial.

Término	Texto original	Corrección	Fuente
Cobertura	"zonas o radio en kilómetros"	Solo zonas de un catálogo jerárquico (ciudad → localidad/comuna → barrio); nunca radio ni geolocalización. Granularidad del MVP: localidad/comuna.	REST-01, ADR-0011

Término	Texto original	Corrección	Fuente
PostGIS	"buscar aliados cercanos por GPS"	No se usa en el MVP. Solo aplica si algún día hay que resolver la zona a partir de coordenadas (ADR-0011 §6); en ese caso es la opción recomendada (100 % de precisión, 7,17 ms p95).	ADR-0011, PoC-004
Backoffice	"personal interno de la plataforma"	Hay dos consolas distintas: la del Administrador del tenant (aprueba aliados, configura categorías y tarifas) y la del Administrador de plataforma (alta y estado de tenants).	SRS §2.3
Ciclo del servicio	"...hasta el pago y la calificación final"	En el MVP termina en calificación y cierre ; el pago corresponde al 2º incremento.	SRS §1.2
Microservicios	"...cada uno con su propia base de datos lógica"	En MANI hay un solo esquema , del que es dueño el Core. Reglas (Java) no tiene base propia y Despacho (.NET) solo ejecuta RPC atómicas.	SDD V1 §12
Aliado	"proveedor independiente o empresa"	Tres tipos: persona natural, empresa o empleado directo del tenant.	RF-05

NEW 14.5 Términos de negocio agregados

- **Administrador de plataforma:** opera MANI como SaaS; da de alta los tenants y administra su estado.
- **Administrador del tenant:** dueño operativo de la empresa suscrita; configura sus reglas y aprueba aliados.
- **Empleado directo:** aliado que pertenece a la nómina del tenant y no pasa por flujo de aprobación.
- **Sitio:** dirección o sede de un cliente. Tiene una zona asignada y puede tener reglas contextuales (p. ej. "traer botas de seguridad").
- **Zona:** elemento del catálogo administrativo global (ciudad, localidad/comuna, barrio). Se desactiva, nunca se elimina.
- **Categoría de servicio:** tipo de trabajo que ofrece un tenant (p. ej. Plomería), con su flujo operativo.
- **Solicitud:** pedido de servicio que crea un cliente desde un sitio y una categoría.
- **Despacho (broadcast):** envío simultáneo de una solicitud a todos los aliados válidos; la primera aceptación válida gana.
- **Aliados válidos:** aliados cuya cobertura coincide exactamente con la zona del sitio y que atienden la categoría.
- **Cotización:** propuesta del aliado con mano de obra y materiales separados. Cada ajuste crea una nueva versión.
- **Tarifa de referencia:** valores mínimo, típico y máximo por categoría y tenant.
- **KYC (Know Your Customer):** documentos de verificación de identidad del aliado, configurables por tenant.
- **Evento del servicio (log):** registro cronológico de solo inserción (append-only) de lo ocurrido en un servicio.
- **Calificación bidireccional:** cliente y aliado se califican mutuamente; el servicio no cierra hasta que ambos lo hayan hecho.

NEW 14.6 Términos técnicos agregados

- **Claim / app_metadata :** dato firmado dentro del JWT. MANI usa `app_metadata.tenant_id` y `app_metadata.user_role` como única fuente del tenant.

- **custom_access_token_hook**: función de PostgreSQL que Supabase Auth invoca al emitir el token para inyectar los claims de tenant y rol. Es *fail-closed*.
- **Tenant spoofing**: suplantación de tenant mediante un valor editable (p. ej. una cabecera `X-Tenant-ID`). Prevenido por ADR-0018.
- **X-Tenant-Slug**: cabecera que solo se usa antes de autenticar, para resolver la empresa en el login.
- **Token relay**: reenvío del JWT del usuario en cada llamada entre servicios, en lugar de usar tokens de servicio.
- **JWKS**: conjunto de claves públicas con el que se verifica la firma de los JWT (ES256).
- **service_role key**: llave de Supabase que omite RLS; solo puede estar en el Core.
- **SECURITY DEFINER**: función SQL que se ejecuta con los permisos de su dueño y omite RLS; debe tomar el tenant de `auth.jwt()`.
- **UPDATE condicional (compare-and-swap)**: actualización que solo afecta la fila si sigue en el estado esperado. Es el mecanismo de exclusión del despacho (ADR-0021).
- **Idempotencia / Idempotency-Key**: repetir una operación produce el mismo efecto que hacerla una sola vez. Se implementa con esa cabecera y con restricciones `UNIQUE`.
- **URL firmada**: enlace temporal a un archivo privado de Storage. Funciona como token al portador, por eso se le pone un TTL corto.
- **PostgREST / Data API**: capa REST que Supabase genera sobre PostgreSQL.
- **Supabase Realtime (Broadcast)**: canal WebSocket para mensajes y eventos casi en tiempo real.
- **FCM / APNs**: servicios de notificaciones push de Android e iOS.
- **Serverpod**: framework de backend en Dart; tecnología del Backend Core.
- **Motor de Reglas**: servicio Java que evalúa las reglas por tenant (ranking, KYC, tarifario). Borrador ADR-0028.
- **Motor de Despacho**: servicio .NET que crea solicitudes y resuelve la aceptación concurrente. Borrador ADR-0028.
- **Migración**: script SQL versionado, numerado e idempotente en `database/migrations/`.
- **GHCR**: GitHub Container Registry, el registro de imágenes Docker del proyecto.
- **Railway**: plataforma de hosting sin costo fijo (ADR-0023).
- **k3d / kind**: herramientas para correr un clúster Kubernetes local. Propuestas para cumplir PROY-08 (ADR-0029, borrador).
- **Kustomize (base / overlays)**: forma de reutilizar manifiestos de Kubernetes con variaciones por ambiente.
- **NetworkPolicy default deny**: regla de Kubernetes que bloquea todo el tráfico entre pods salvo lo permitido explícitamente.
- **Gitflow / rama release**: modelo de ramas `develop → release → main`; en MANI `release` es persistente.
- **Build once, deploy anywhere**: se promueve la misma imagen entre ambientes, sin recompilar.
- **SAST / DAST**: análisis de seguridad estático (SonarCloud) y dinámico (OWASP ZAP).
- **Quality Gate**: umbral de calidad de SonarCloud que bloquea el merge si no se cumple.



14.7 Términos de arquitectura y metodología agregados

- **ADR (Architecture Decision Record)**: registro de una decisión de arquitectura con su contexto, alternativas, decisión, trade-off y consecuencias.
- **Mesa de Arquitectura**: órgano colegiado del equipo que ratifica las decisiones (quórum 5 de 7, ADR-0003).

- **Driver (DR) / Killer (KI):** objetivo que la arquitectura debe lograr / riesgo que puede invalidarla.
- **Atributo de calidad (AC):** característica de ISO/IEC 25010 (seguridad, fiabilidad, etc.).
- **Escenario de calidad (QS):** estímulo, respuesta y medida que vuelven verificable un atributo.
- **Trade-off (TO):** tensión entre dos escenarios y la decisión tomada frente a ella.
- **PoC (prueba de concepto):** verifica una decisión ya redactada, con umbral publicado antes de medir y control negativo.
- **Spike:** investigación con timebox para responder una pregunta abierta antes de decidir.
- **Control negativo:** variante del experimento que debe fallar; si no puede fallar, la prueba no valida nada.
- **Benchmarking:** comparación medida entre alternativas (p. ej. PostGIS vs bounding box vs geohash).
- **Modelo 4+1:** cinco vistas de arquitectura (lógica, procesos, desarrollo, física y escenarios).
- **C4 model:** notación de diagramas en cuatro niveles (contexto, contenedores, componentes y código).
- **BPMN / MER:** notación de procesos de negocio / modelo entidad-relación.
- **DoR / DoD:** Definition of Ready (paso a QA, ADR-0026) / Definition of Done (pase a `main`, ADR-0025).
- **Gate:** conjunto de criterios que deben cumplirse para promover entre ambientes.